Software Requirements Specification (SRS)

SmartCart Distributed E-Commerce Platform

Document Version: 1.0.0
Date: October 16, 2026
Status: Approved / Production Specification
Architecture: Polyglot Microservices Architecture
Group / Team: Group 03 (TeamID 03)
Repository: https://github.com/Ayush17314/SmartCart-Group-03
Deliverables Drive: https://drive.google.com/drive/folders/1q9IY8r2_aUCeogiUsT0tpODwt3hF6PHL?usp=sharing

Table of Contents

1. Introduction
   - 1.1 Purpose
   - 1.2 Document Conventions
   - 1.3 Intended Audience
   - 1.4 Project Scope
   - 1.5 References & Standards
2. Overall Description
   - 2.1 Product Perspective & Context
   - 2.2 System Architecture Diagram
   - 2.3 User Classes and Characteristics (RBAC)
   - 2.4 Operating Environment & Port Allocations
   - 2.5 Design and Implementation Constraints
   - 2.6 Assumptions and Dependencies
3. System Features & Microservices Requirements
   - 3.1 API Gateway Service (Port 4000)
   - 3.2 Authentication & Identity Service (Port 4001)
   - 3.3 Product Catalog & Inventory Service (Port 4002)
   - 3.4 Shopping Cart Service (Port 4003)
   - 3.5 Wishlist Service (Port 4004)
   - 3.6 Order Orchestration & Promotions Service (Port 4005)
   - 3.7 Payment & Reconciliation Service (Port 4006)
   - 3.8 Notification & Communication Service (Port 4007)
   - 3.9 Storefront & Management Web Client (Port 3000)
4. External Interface Requirements
   - 4.1 User Interfaces
   - 4.2 Hardware & Network Interfaces
   - 4.3 Software & Cloud Interfaces
   - 4.4 Communications Interfaces & Protocols
5. Non-Functional Requirements (NFRs)
   - 5.1 Security & Cryptography

- 5.2 Reliability, Retries & Fault Tolerance
- 5.3 Performance & Caching
- 5.4 Observability, Logging & Auditability
- 5.5 Maintainability, Containerization & Compliance

6. Data Models & Persistence Architecture
   - 6.1 Database Isolation Model
   - 6.2 Relational Schema (MySQL)
   - 6.3 Document Schemas (MongoDB)
7. Appendix & Traceability Matrix
   - 7.1 Status Code Mapping
   - 7.2 Problem Details Error Catalogue
   - 7.3 Assignments 1–6 Traceability Verification

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a rigorous, complete, and verifiable description of the **SmartCart** distributed e-commerce platform. It details all functional capabilities, microservice boundaries, data contracts, security mechanisms, external partner integrations, and quality attributes implemented in the production codebase.

1.2 Document Conventions

- **RFC 2119 Keywords:** The keywords **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **RECOMMENDED**, and **MAY** are used in accordance with BCP 14, RFC 2119.
- **Identifiers:** Functional requirements are prefixed with `FR-[SERVICE]-[ID]`. Non-functional requirements are prefixed with `NFR-[CATEGORY]-[ID]`.
- **API Payloads:** Standard request and response bodies use JSON (`application/json`) with RFC 7807/9457 Problem Details (`application/problem+json`) for error states.

1.3 Intended Audience

This specification is designed for software architects, backend engineers, frontend developers, DevOps personnel, academic evaluators, and system auditors reviewing the system against standard enterprise criteria and coursework assignments (Assignments 1 through 6).

1.4 Project Scope

SmartCart is an Amazon-inspired e-commerce platform engineered with a **polyglot microservices backend** and a modern **Next.js 15 App Router frontend**. The platform supports full e-commerce lifecycles: catalog discovery, text search, shopping cart sessions, multi-role access control, checkout orchestration with stock reservation, online payment processing via Razorpay (UPI, Credit/Debit cards), Cash on Delivery (COD), order state machines, return authorization, transactional notifications, and admin analytics.

1.5 References & Standards

- **RFC 7230–7235:** Hypertext Transfer Protocol (HTTP/1.1) Semantics and Quality Attributes
- **RFC 7807 / RFC 9457:** Problem Details for HTTP APIs (`application/problem+json`)

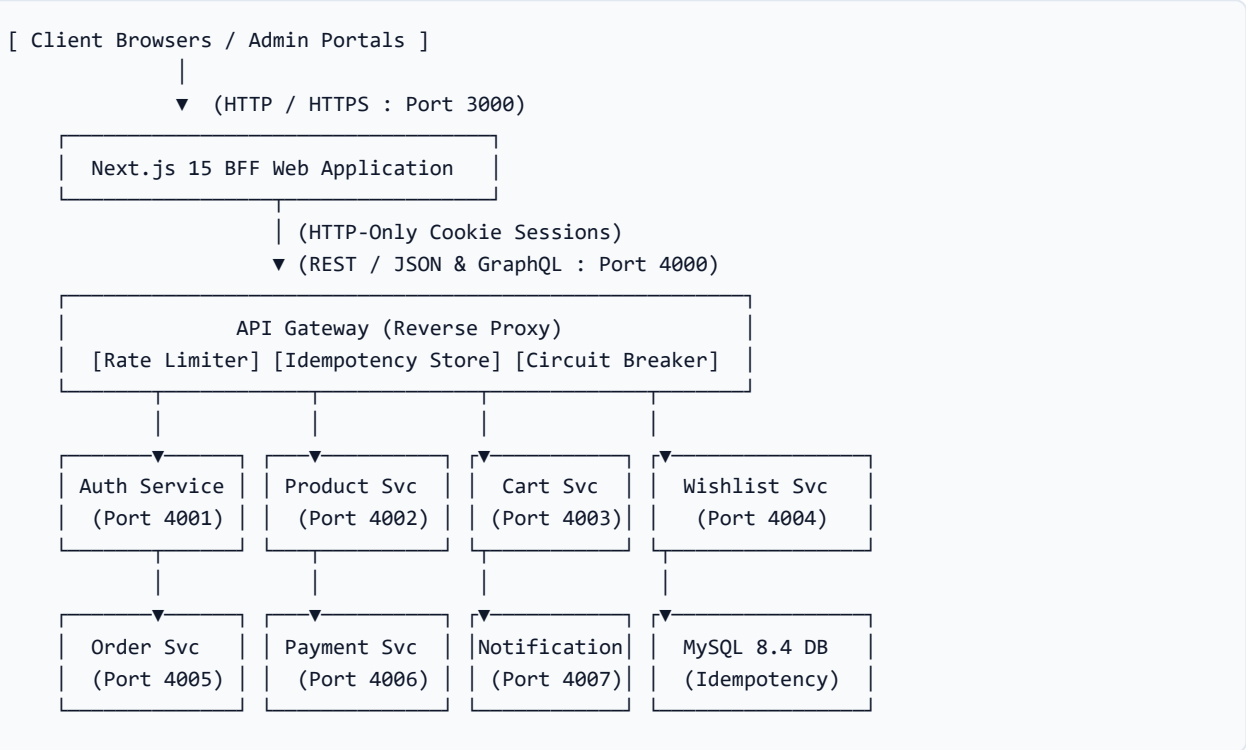
- **RFC 7519:** JSON Web Token (JWT) Architecture
- **RFC 5988:** Web Linking (Pagination `Link` headers)
- **OpenAPI 3.0.3:** Specification for RESTful API Contracts
- **IEEE 830-1998 / ISO/IEC/IEEE 29148:** Recommended Practice for Software Requirements Specifications

---

2. Overall Description

2.1 Product Perspective & Context

SmartCart operates as a decentralized, service-oriented architecture. Individual microservices own their database persistence models and business logic without cross-service database access. All cross-domain interaction is mediated through HTTP/REST endpoints, asynchronous webhooks, and the centralized API Gateway.



2.2 System Architecture & Port Allocations

Component	Port	Runtime / Framework	Primary Data Store	Primary Responsibility
Frontend Web App	`3000`	Next.js 15, React 19, Tailwind v4	Browser Session / Cookie	Storefront, Admin Suite, Staff Portal, Customer Account
API Gateway	`4000`	Node.js, Express, Helmet, AJV	MySQL 8.4 (`smartcart`)	Reverse Proxy, Rate Limiting, Idempotency, OpenAPI, GraphQL
Auth Service	`4001`	Node.js, Express, Mongoose	MongoDB (`smartcart_auth`)	User accounts, Google OAuth, RBAC permissions, Staff invites

Component	Port	Runtime / Framework	Primary Data Store	Primary Responsibility
Product Service	`4002`	Node.js, Express, Mongoose	MongoDB (`smartcart_products`)	Catalog search, categories, inventory stock history, reviews
Cart Service	`4003`	Node.js, Express, Mongoose	MongoDB (`smartcart_cart`)	Customer cart items, server-side subtotals
Wishlist Service	`4004`	Node.js, Express, Mongoose	MongoDB (`smartcart_wishlist`)	User saved items, move-to-cart operations
Order Service	`4005`	Node.js, Express, Mongoose	MongoDB (`smartcart_orders`)	Order checkout orchestration, coupons, shipment tracking
Payment Service	`4006`	Node.js, Express, Mongoose	MongoDB (`smartcart_payments`)	Razorpay orders, HMAC-SHA256 signature verification, COD
Notification Service	`4007`	Node.js, Express, Mongoose	MongoDB (`smartcart_notifications`)	Transactional email delivery (Resend / Mock), event audit
MySQL Database	`3306`	MySQL 8.4 Community	InnoDB Tables	Distributed idempotency keys, persistent request audits
MongoDB Instance	`27017`	MongoDB 7.0 / Atlas	Document Collections	Domain microservice databases

2.3 User Classes and Characteristics (RBAC Hierarchy)

The system enforces a strict Role-Based Access Control model defined by permissions issued in cryptographically signed JWT tokens:

1. `customer` **(Default Storefront User):**
   - Access: `/login` , `/register` , `/account` , `/cart` , `/checkout`
   - Permissions: `products.read` , `orders.read_own` , `refunds.request`
   - Capabilities: Product search, add to cart/wishlist, place orders via Razorpay or COD, track order shipments, request return within 30 days of delivery, submit reviews.

2. `support` **(Customer Service Representative):**
   - Access: Staff Portal (`/admin/login`)
   - Permissions: `products.read` , `orders.read_all` , `customers.read` , `returns.review` , `refunds.request` , `notifications.manage`
   - Capabilities: Search and view customer order histories, evaluate return requests (approve or reject with justification), queue refund authorizations.

3. `inventory_staff` **(Warehouse / Catalog Operator):**
   - Access: Staff Portal (`/admin/login`)
   - Permissions: `products.read` , `products.write` , `inventory.read` , `inventory.write` , `orders.read_all`
   - Capabilities: Create and edit catalog listings, modify prices, perform inventory stock adjustments with mandatory logged audit reasons, inspect low-stock alerts.

4. `admin` **(Store Operations Manager):**
   - Access: Admin Management Suite (`/admin`)
   - Permissions: All staff capabilities plus `orders.manage` , `customers.manage` , `promotions.manage` ,

`reviews.moderate`, `staff.read`, `analytics.read`
- Capabilities: Advance order states (`CONFIRMED` $\rightarrow$ `SHIPPED` $\rightarrow$ `DELIVERED`), assign carrier tracking numbers, create/modify coupon discount codes, moderate customer reviews.

5. `super_admin` **(Platform Administrator):**
- Access: Super Admin Governance Suite (`/super-admin`)
- Permissions: Complete system wildcard (`*`), including `refunds.approve`, `staff.manage`, `settings.manage`, `audit.read`
- Capabilities: Protected by multi-step login challenges; provisions staff invitations; authorizes financial refunds through Razorpay; configures taxes, shipping rules, and store settings; reviews immutable system audits.

2.4 Operating Environment & Port Allocations

- **Server OS:** Ubuntu 22.04 LTS / Windows 10/11 / macOS Darwin
- **Container Engine:** Docker Engine v24.0+ with Docker Compose v2.20+
- **Runtimes:** Node.js v20.x or v22.x LTS, Python 3.10+ (scripts)
- **Database Servers:** MongoDB Community Server 7.0.x and MySQL 8.4.x

2.5 Design and Implementation Constraints

- **Zero Shared Databases:** No microservice SHALL query or modify another microservice's MongoDB database.
- **Backend-for-Frontend (BFF) Security:** Browser clients SHALL NOT receive raw JWT bearer tokens directly. The Next.js BFF server SHALL manage session tokens within secure, HTTP-only, SameSite cookies.
- **Backward Compatibility:** All error payloads MUST deliver RFC 7807/9457 Problem Details while preserving legacy `success: false` and `message` properties for frontend stability.

---

3. System Features & Microservices Requirements

3.1 API Gateway Service (Port 4000)

FR-GW-01: Reverse Proxy & URL Rewriting

The API Gateway SHALL route incoming HTTP requests to target upstream microservices based on route prefixes. The Gateway SHALL support both `/api/*` and versioned `/v1/api/*` routes, stripping the `/v1` prefix transparently before proxying.

FR-GW-02: Rate Limiting & Retry Semantics

The Gateway SHALL enforce IP-based and client-based rate limiting via `express-rate-limit`.
- Default quota: 120 requests per 60-second sliding window (`RATE_LIMIT_MAX=120`).
- Exceeded quotas MUST return HTTP status `429 Too Many Requests` with `Content-Type: application/problem+json`.
- The response MUST include standard headers: `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After: <seconds>`.

FR-GW-03: Durable Idempotency on Mutating Requests

For state-changing `POST` requests (`/api/orders`, `/api/products`, `/api/cart/items`, `/api/payments/create`, and returns), the client MUST supply an `Idempotency-Key` header (1–200 ASCII characters).
- The Gateway SHALL compute a SHA-256 fingerprint of the request payload and headers.
- Identical replays within the retention period SHALL return the cached response with header `Idempotency-`

`Replayed: true`.
- Key reuse with a mismatched request payload MUST be rejected with HTTP status `409 Conflict`.
- State storage MUST be backed by MySQL table `api_idempotency` with automatic fallback to memory in test environments.

FR-GW-04: Schema Validation at Boundary

The Gateway SHALL pre-validate incoming JSON bodies using AJV schemas (`registration`, `login`, `productCreate`, `orderCreate`, `cartItem`, `paymentCreate`, `superAdminCode`, `emailToken`).
- Malformed payloads MUST be rejected at the Gateway with HTTP `400 Bad Request`.
- Validation errors MUST return an `errors[]` array listing all failing properties simultaneously (`field`, `message`, `keyword`).

FR-GW-05: Outbound Network Resilience

Downstream microservice requests from the Gateway SHALL be protected by:
- Strict timeouts using `AbortSignal.timeout` (default 5000ms).
- Exponential backoff retries (restricted to safe, idempotent `GET` and `HEAD` requests).
- A stateful Circuit Breaker (`resilience.js`). Upon reaching 5 consecutive failures, the breaker SHALL open for 15,000ms, returning HTTP `503 Service Unavailable` with `Retry-After: 15`.

FR-GW-06: Content Negotiation & OPTIONS Verification

- Incoming requests with unsupported `Accept` headers SHALL be rejected with HTTP `406 Not Acceptable`. Supported media types include `application/json`, `application/xml`, and `text/xml`.
- The Gateway SHALL handle HTTP `OPTIONS` requests for all major resources, returning HTTP `204 No Content` and an explicit `Allow` header listing supported HTTP methods.

FR-GW-07: Single-Endpoint GraphQL

The Gateway SHALL expose `POST /graphql` executing against a unified catalog schema allowing clients to query products, filter by category/search, and retrieve cursor-paginated results.

3.2 Authentication & Identity Service (Port 4001)

FR-AUTH-01: User Registration & Password Hashing

The service SHALL register users with name, email, password, and phone. Passwords MUST be hashed using `bcryptjs` with a work factor ≥ 10. Plaintext passwords SHALL NOT be stored or logged.

FR-AUTH-02: Multi-Portal Authentication

The service SHALL provide isolated authentication endpoints for distinct user classes:
- `POST /api/auth/customer-login` (Restricted to `customer` role)
- `POST /api/auth/staff-login` (Restricted to `support`, `inventory_staff`, `admin`)
- `POST /api/auth/super-admin-login` (Restricted to `super_admin`)
- `POST /api/auth/super-admin-verify` (Step-2 challenge code validation)

FR-AUTH-03: BFF Token Secrecy

The Auth Service SHALL only release the raw JWT token in the response body if the request contains a valid, matching `X-Auth-Exchange-Secret` header supplied by the trusted Next.js server. Public browser logins through the gateway receive only sanitized user metadata.

FR-AUTH-04: Session Revocation via Token Versioning

User records in MongoDB SHALL track an integer `tokenVersion`. When an administrator revokes sessions or a user changes their password, `tokenVersion` SHALL increment, instantly invalidating all existing signed JWTs.

FR-AUTH-05: Password Reset Flow

The service SHALL support self-service password recovery:
- `POST /api/auth/forgot-password` generates a cryptographically secure 32-byte token, stores its SHA-256 hash with a 30-minute expiration, and queues a password reset email.
- `POST /api/auth/reset-password` validates the token hash and updates the password.

FR-AUTH-06: Staff Invitation Management

Super Administrators SHALL generate invitation tokens (`POST /api/auth/invite-staff`) specifying target email, assigned role, and permissions. The service sends an invitation email; the recipient activates their account via `POST /api/auth/accept-invite`.

3.3 Product Catalog & Inventory Service (Port 4002)

FR-PROD-01: Catalog Search & Navigation

The service SHALL expose `GET /api/products` supporting full-text search (`search=`), category filtering (`category=`), brand filtering, and price range constraints.

FR-PROD-02: Cursor-Based & Offset Pagination

The catalog SHALL support stable cursor-based pagination (`first=20` , `after=<cursor>`). The response MUST return `pagination.nextCursor` and RFC 5988 `Link: <...>; rel="next"` headers alongside classic `page=` and `limit=` query support.

FR-PROD-03: Conditional Caching (ETag / 304)

Single product requests (`GET /api/products/:id`) SHALL compute an MD5 hash of the resource ID, update timestamp, stock, and price, returning an `ETag` header. If the client supplies a matching `If-None-Match` header, the service MUST respond with HTTP `304 Not Modified` and an empty payload.

FR-PROD-04: Optimistic Concurrency Control (If-Match / 412)

Product updates via `PUT /api/products/:id` SHALL inspect the `If-Match` header. If the client ETag does not match the current product state, the service MUST reject the mutation with HTTP `412 Precondition Failed`.

FR-PROD-05: Audited Inventory Stock Adjustments

Stock modifications MUST require an `actorUserId` and a mandatory explanation string (`stockAdjustmentReason` ≥ 3 characters). The service SHALL append an immutable audit record to `stockHistory[]` containing timestamp, actor, previous stock, new stock, and reason.

FR-PROD-06: Customer Reviews & Ratings

Verified customers SHALL submit product ratings (1–5 stars) and text comments (`POST /api/reviews`). Administrators SHALL have permission to approve, hide, or moderate submitted reviews.

3.4 Shopping Cart Service (Port 4003)

FR-CART-01: Persistent Shopping Cart Management

The Cart Service SHALL maintain a persistent cart document in MongoDB per authenticated user (`GET /api/cart`).

FR-CART-02: Item Operations

Customers SHALL add products (`POST /api/cart/items`), update item quantities (`PUT /api/cart/items/:productId`), and remove items (`DELETE /api/cart/items/:productId`). Quantities MUST be positive integers (≥ 1).

FR-CART-03: Server-Side Subtotal Verification

The Cart Service SHALL calculate item line totals and cart grand totals on the server side to prevent client price tampering.

3.5 Wishlist Service (Port 4004)

FR-WISH-01: User Wishlist Persistence

The service SHALL manage saved item arrays per user in MongoDB (`smartcart_wishlist`). Customers SHALL add items (`POST /api/wishlist`), remove items (`DELETE /api/wishlist/:productId`), and inspect saved items (`GET /api/wishlist`).

FR-WISH-02: Move to Cart Integration

The service SHALL support moving a wishlist item directly into the customer's active cart in a single user action.

3.6 Order Orchestration & Promotions Service (Port 4005)

FR-ORD-01: Central `placeOrder` Orchestration

The Order Service SHALL coordinate checkout transactions:
1. Validate that the cart items array is not empty (return HTTP `422 Unprocessable Entity` with `type: "urn:smartcart:error:empty-cart"` if empty).
2. Query Product Service to verify current prices and verify inventory availability.
3. Reserve product stock via internal Product Service calls.
4. Evaluate and apply promotional coupon discounts.
5. Compute sales tax and shipping fees.
6. Invoke Payment Service (`POST /api/payments/create`) to create a payment intent.
7. Clear customer cart upon order creation.
8. If downstream steps fail, execute compensatory rollbacks (release reserved inventory).
9. Return HTTP `201 Created` with a canonical `Location: /api/orders/:id` header.

FR-ORD-02: State Machine Lifecycle

Orders SHALL advance strictly through defined status transitions:
`PENDING` $\rightarrow$ `CONFIRMED` $\rightarrow$ `SHIPPED` $\rightarrow$ `DELIVERED`
`PENDING` / `CONFIRMED` $\rightarrow$ `CANCELLED`
`DELIVERED` $\rightarrow$ `RETURNED` (within 30 days)
Illegal status transitions MUST be rejected with HTTP `409 Conflict` .

FR-ORD-03: Optimistic Concurrency on Updates

Order status changes (`PATCH /api/orders/:id/status`) and cancellations (`POST /api/orders/:id/cancel`) SHALL validate client `If-Match` ETags, returning HTTP `412 Precondition Failed` on stale requests.

FR-ORD-04: Promotion & Coupon Engine

The service SHALL evaluate promo codes (`/api/promotions`). Discounts SHALL support percentage reductions (`PERCENTAGE`) and fixed rupee amounts (`FIXED`), subject to expiration dates, minimum order thresholds, and per-user usage limits.

3.7 Payment & Reconciliation Service (Port 4006)

FR-PAY-01: Razorpay Order Creation

For online payment methods (`UPI` , `CARD`), the service SHALL initialize a payment order via Razorpay REST API in Indian Rupee subunits (Paise).

FR-PAY-02: HMAC-SHA256 Signature Verification

Upon payment completion in the client, the service SHALL verify the cryptographic signature (`POST /api/payments/verify/:paymentId`):
$$\text{Expected Signature} = \text{HMAC-SHA256}(\text{order_id} + "|" + \text{payment_id}, \text{RAZORPAY_KEY_SECRET})$$
Mismatched signatures MUST immediately transition the payment status to `FAILED` .

FR-PAY-03: Webhook Event Processing

The service SHALL expose `POST /api/payments/webhook` with raw body signature validation to process asynchronous provider events (`payment.captured` , `payment.failed` , `refund.processed`).

FR-PAY-04: Cash on Delivery (COD) Flow

The service SHALL support offline COD orders, creating a pending payment record that transitions to `SUCCESS` upon verified courier delivery.

FR-PAY-05: Automated Refunds

Super Administrators SHALL issue payment refunds (`POST /api/payments/refund/:paymentId`). Approved refunds for online transactions SHALL initiate automated refunds via Razorpay API and sync order status.

3.8 Notification & Communication Service (Port 4007)

FR-NOTIF-01: Transactional Email Delivery

The service SHALL deliver transactional emails triggered by internal service events:
- Account verification and password recovery
- Order confirmation with itemized receipt
- Shipping dispatch with tracking link
- Super Admin 2-step verification challenge codes

FR-NOTIF-02: Provider Decoupling & Mock Fallback

The service SHALL support live email delivery via **Resend API** and automatic local fallback to a structured mock console logger when external API keys are not supplied in development/testing.

3.9 Storefront & Management Web Client (Port 3000)

FR-UI-01: Multi-Persona Portals

The Next.js 15 web client SHALL provide tailored user interfaces:
- Storefront catalog, product details, interactive cart, and multi-step checkout.
- Customer account dashboard with order tracking and return forms.
- Staff portal for inventory management and support tickets.
- Admin dashboard with order dispatch workflows and analytics charts.
- Super Admin governance suite for staff provisioning and refund approvals.

FR-UI-02: Resilient Client Querying

Client data fetching SHALL use `@tanstack/react-query` with optimistic cache updates, automatic background refetching, and graceful error boundary handling.

---

4. External Interface Requirements

4.1 User Interfaces

- Responsive Web Design supporting desktop ($\ge 1280\text{px}$), tablet ($768\text{px} - 1024\text{px}$), and mobile devices ($\le 640\text{px}$).
- Accessibility conforming to WCAG 2.1 Level AA standards.

4.2 Hardware & Network Interfaces

- Standard TCP/IP networking across internal Docker bridge networks (`smartcart-network`).
- External client traffic over HTTPS (Port 443) terminating at reverse proxy/gateway.

4.3 Software & Cloud Interfaces

- **Razorpay Payments API:** HTTPS REST API (`https://api.razorpay.com/v1`)
- **Resend Email API:** HTTPS REST API (`https://api.resend.com/emails`)
- **Google OAuth 2.0:** OpenID Connect identity endpoints
- **MongoDB Atlas / Community:** Connection string via `mongodb://` or `mongodb+srv://`
- **MySQL 8.4 Server:** Native connection protocol via `mysql2` driver

4.4 Communications Interfaces & Protocols

- **Client $\leftrightarrow$ Gateway:** HTTP/1.1 and HTTP/2 over TLS, JSON (`application/json`), Problem Details (`application/problem+json`), and XML (`application/xml`).
- **Gateway $\leftrightarrow$ Microservices:** HTTP REST with internal security tokens and correlation headers (`X-Request-Id` , `X-Auth-Exchange-Secret` , `X-Payment-Service-Secret`).

---

5. Non-Functional Requirements (NFRs)

5.1 Security & Cryptography

- **NFR-SEC-01 (Password Security):** Passwords MUST be hashed with `bcrypt` using at least 10 salt rounds.
- **NFR-SEC-02 (Token Confidentiality):** JWTs MUST be signed using HMAC-SHA256 with secrets ≥ 256 bits. Browser cookies MUST carry `HttpOnly`, `SameSite=Lax` (or `Strict`), and `Secure` flags.
- **NFR-SEC-03 (Timing Attacks):** Verification of internal exchange secrets and signature hashes MUST use `crypto.timingSafeEqual()`.
- **NFR-SEC-04 (Header Hardening):** API Gateway MUST enforce HTTP security headers via `helmet`: `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and disabled `X-Powered-By`.
- **NFR-SEC-05 (CORS Security):** CORS MUST explicitly validate origin allowlists; wildcard `*` origins with credentials SHALL NOT be permitted.

5.2 Reliability, Retries & Fault Tolerance

- **NFR-REL-01 (Circuit Breaking):** Upstream service failures exceeding 5 faults in 15 seconds SHALL open the circuit breaker, failing fast without thread starvation.
- **NFR-REL-02 (Safe Retries):** Retries with exponential backoff SHALL only be executed on idempotent HTTP methods (`GET`, `HEAD`). Mutating `POST` / `PATCH` calls SHALL NEVER be automatically retried without idempotency keys.
- **NFR-REL-03 (Idempotency Retention):** Completed idempotent responses SHALL be retained in MySQL for at least 24 hours.

5.3 Performance & Caching

- **NFR-PERF-01 (Gateway Overhead):** Gateway proxy latency SHALL NOT exceed 15ms under normal load (excluding upstream service duration).
- **NFR-PERF-02 (Bandwidth Conservation):** Unmodified resources MUST return HTTP `304 Not Modified` with zero-length response bodies when `If-None-Match` matches.
- **NFR-PERF-03 (Payload Boundaries):** JSON body parsing limits SHALL be strictly capped (Gateway: 200KB, Microservices: 100KB) to prevent Denial of Service (DoS) attacks.

5.4 Observability, Logging & Auditability

- **NFR-OBS-01 (Correlation Stamping):** All incoming HTTP requests SHALL be assigned an `X-Request-Id` UUID at the Gateway, propagated through all internal service hops.
- **NFR-OBS-02 (Structured JSON Logs):** All server logs MUST emit single-line machine-readable JSON containing `level`, `service`, `requestId`, `method`, `path`, `status`, and `durationMs`.
- **NFR-OBS-03 (Security Audit Trail):** State-changing actions (authentications, staff modifications, refunds, inventory overrides) MUST be written to immutable audit collections.

5.5 Maintainability & Standards Compliance

- **NFR-MAINT-01 (RFC 7807/9457 Compliance):** Error responses MUST conform to standard Problem Details schemas.
- **NFR-MAINT-02 (Twelve-Factor Configuration):** All database URIs, port bindings, timeouts, and cryptographic secrets MUST be injected via environment variables.

---

6. Data Models & Persistence Architecture

6.1 Database Isolation Model

SmartCart enforces strict multi-database isolation:

```
┌──────────────────────────────────────────────────────┐
│                      DATABASES                       │
├──────────────────────────┬───────────────────────────┤
│ MySQL 8.4:               │ MongoDB Collections:      │
│ • smartcart              │ • smartcart_auth          │
│   ├── api_idempotency    │ • smartcart_products      │
│   └── api_request_audit  │ • smartcart_cart          │
│                          │ • smartcart_wishlist      │
│                          │ • smartcart_orders        │
│                          │ • smartcart_payments      │
│                          │ • smartcart_notifications │
└──────────────────────────┴───────────────────────────┘
```

6.2 Relational Schema (MySQL `smartcart`)

Table: `api_idempotency`

```
CREATE TABLE IF NOT EXISTS api_idempotency (
    idempotency_key VARCHAR(255) PRIMARY KEY,
    route_path VARCHAR(255) NOT NULL,
    method VARCHAR(16) NOT NULL,
    request_hash CHAR(64) NOT NULL,
    status_code INT NOT NULL,
    response_headers JSON NOT NULL,
    response_body LONGTEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Table: `api_request_audit`

```
CREATE TABLE IF NOT EXISTS api_request_audit (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    request_id VARCHAR(64) NOT NULL,
    route_path VARCHAR(255) NOT NULL,
    method VARCHAR(16) NOT NULL,
    status_code INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_request_id (request_id),
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3 Document Schemas (MongoDB Mongoose)

Database: `smartcart_auth`

- `users` Collection:
- `name`: String, required, maxlength 200

- `email` : String, required, unique, indexed, lowercase
- `password` : String, required, select: false
- `role` : Enum (`customer` , `support` , `inventory_staff` , `admin` , `super_admin`)
- `isActive` : Boolean, default true
- `emailVerified` : Boolean, default false
- `googleId` : String, indexed
- `tokenVersion` : Number, default 0
- `addresses` : Array of Address objects
- `passwordResetTokenHash` : String, select: false, expires Date
- `staffInviteTokenHash` : String, select: false, expires Date
- **`authauditevents` Collection:**
- `actorUserId` , `action` , `targetUserId` , `outcome` , `requestId` , `metadata` , `createdAt`

Database: `smartcart_products`

- **`products` Collection:**
- `name` : String, required, indexed
- `description` : String
- `price` : Number, required, min 0
- `category` : String, required, indexed
- `brand` : String
- `stock` : Number, required, min 0
- `images` : Array of HTTPS URL strings
- `stockHistory` : Array of `{ actorUserId, previousStock, newStock, reason, changedAt }`
- `ratingAverage` : Number, default 0
- `ratingCount` : Number, default 0
- `archivedAt` : Date, default null
- **`reviews` Collection:**
- `productId` , `userId` , `rating` (1–5), `comment` , `isModerated` , `createdAt`

Database: `smartcart_orders`

- **`orders` Collection:**
- `userId` : String, required, indexed
- `items` : Array of `{ productId, name, price, quantity, imageUrl }`
- `subtotal` : Number, required
- `discountAmount` : Number, default 0
- `shippingFee` : Number, default 0
- `taxAmount` : Number, default 0
- `totalAmount` : Number, required
- `couponCode` : String, default null
- `shippingAddress` : Object `{ fullName, street, city, state, postalCode, phone }`
- `orderStatus` : Enum (`PENDING` , `CONFIRMED` , `SHIPPED` , `DELIVERED` , `CANCELLED` , `RETURNED`)
- `paymentStatus` : Enum (`PENDING` , `SUCCESS` , `FAILED` , `REFUNDED`)
- `paymentMethod` : Enum (`UPI` , `CARD` , `COD`)

- `trackingNumber` : String
- `carrier` : String
- `returnReason` : String
- `createdAt` , `updatedAt`
- **`promotions`** **Collection:**
- `code` : String, required, unique, uppercase
- `discountType` : Enum (`PERCENTAGE` , `FIXED`)
- `discountValue` : Number, required
- `minOrderAmount` : Number, default 0
- `maxDiscountAmount` : Number
- `usageLimit` : Number
- `usedCount` : Number, default 0
- `expiresAt` : Date
- `isActive` : Boolean, default true

Database: **`smartcart_payments`**

- **`payments`** **Collection:**
- `orderId` : String, required, unique, indexed
- `userId` : String, required, indexed
- `amount` : Number, required
- `currency` : String, default "INR"
- `paymentMethod` : Enum (`UPI` , `CARD` , `COD`)
- `status` : Enum (`PENDING` , `SUCCESS` , `FAILED` , `REFUNDED`)
- `razorpayOrderId` : String
- `razorpayPaymentId` : String
- `razorpaySignature` : String
- `events` : Array of `{ type, message, timestamp }`
- `refunds` : Array of `{ refundId, amount, status, reason, createdAt }`
- **`paymentaudits`** **Collection:**
- `paymentId` , `orderId` , `actorUserId` , `action` , `outcome` , `details` , `createdAt`

---

7. Appendix & Traceability Matrix

7.1 Status Code Mapping

Status Code	Standard Reason Phrase	SmartCart Context / Trigger
`200`	OK	Successful `GET` , `PUT` , `PATCH` , or idempotent login verification.
`201`	Created	Resource successfully created (`POST /api/orders` , `/api/products`); emits `Location` header.

Status Code	Standard Reason Phrase	SmartCart Context / Trigger
`204`	No Content	Preflight and explicit `OPTIONS` requests; emits `Allow` header.
`304`	Not Modified	Conditional `GET` where request `If-None-Match` matches calculated entity `ETag`.
`400`	Bad Request	AJV schema validation errors; emits Problem Details with `errors[]` array.
`401`	Unauthorized	Missing or invalid JWT bearer token on protected routes.
`403`	Forbidden	User lacks necessary RBAC permission claim for target operation.
`404`	Not Found	Route does not exist or target entity ID not found in database.
`406`	Not Acceptable	Client `Accept` header requests an unsupported media representation.
`409`	Conflict	Duplicate resource SKU, illegal order state transition, or reused Idempotency-Key with changed body.
`412`	Precondition Failed	Optimistic concurrency mismatch: request `If-Match` does not match resource's current `ETag`.
`422`	Unprocessable Entity	Semantically invalid domain payload (e.g. checkout request with empty items array).
`429`	Too Many Requests	Rate limit quota exceeded; emits `Retry-After: <seconds>` header.
`500`	Internal Server Error	Uncaught server exception; safely masked to prevent stack trace leakage.
`503`	Service Unavailable	Downstream microservice offline or circuit breaker tripped open; emits `Retry-After`.
`504`	Gateway Timeout	Downstream HTTP call exceeded `AbortSignal.timeout` duration.

7.2 Problem Details Error Catalogue (`application/problem+json`)

Error URI Type Slug	HTTP Status	Title	Domain Description
`urn:smartcart:error:empty-cart`	`422`	Unprocessable Entity	Checkout initiated with an empty items array.
`urn:smartcart:error:item-unavailable`	`409`	Conflict	Requested product is out of stock or inactive.
`urn:smartcart:error:payment-declined`	`400`	Bad Request	Card/UPI payment authorization failed or signature invalid.
`urn:smartcart:error:order-not-found`	`404`	Not Found	Order ID does not match any existing record.
`urn:smartcart:error:product-not-found`	`404`	Not Found	Product ID does not exist or is archived.

Error URI Type Slug	HTTP Status	Title	Domain Description
`urn:smartcart:error:illegal-transition`	`409`	Conflict	Invalid state machine lifecycle advancement.
`urn:smartcart:error:precondition-failed`	`412`	Precondition Failed	Resource modified concurrently; `If-Match` check failed.
`urn:smartcart:error:rate-limit-exceeded`	`429`	Too Many Requests	Request quota exhausted within active window.
`urn:smartcart:error:not-acceptable`	`406`	Not Acceptable	Requested `Accept` header format is unsupported.
`urn:smartcart:error:validation-error`	`400`	Bad Request	Request payload failed AJV JSON schema constraints.
`urn:smartcart:error:authentication-required`	`401`	Unauthorized	Bearer token missing on protected route.
`urn:smartcart:error:insufficient-permissions`	`403`	Forbidden	User role lacks permission claim.
`urn:smartcart:error:service-unavailable`	`503`	Service Unavailable	Downstream service down or circuit breaker tripped open.

7.3 Assignments 1–6 Traceability Verification

Assignment	Syllabus Requirement	SmartCart Codebase Implementation	Verification Evidence
Assignment 1	Raw HTTP & Project Setup	Modular service directories, Git repo with readable commits, `.gitignore`, `.env.example`.	Repo commit log, clean structure.
Assignment 2	Services, Contracts & Schema	8 microservices, isolated schemas, `placeOrder` orchestration, cross-service HTTP calls.	`order-service/src/server.js`, MongoDB instances.
Assignment 3	External Partner Edge	Payment gateway integration with signatures, webhooks, and refund edge (modeled via REST/JSON).	`payment-service/src/services/payment.service.js`.
Assignment 4	REST Service & Resilience	Plural resources (`/api/orders`), OpenAPI 3.0 spec (`/docs`), `Idempotency-Key` replay (409), circuit breaker, retries.	`api-gateway/src/openapi.js`, `resilience.js`, `sql-store.js`.

Assignment	Syllabus Requirement	SmartCart Codebase Implementation	Verification Evidence
Assignment 5	HTTP Methods & Headers	OPTIONS with `Allow`, dynamic `ETag` (304), `If-Match` concurrency (412), `Accept` negotiation (406), Rate limiting with `Retry-After` (429).	`api-gateway/src/app.js`, `order-service/src/server.js`, `api.test.js`.
Assignment 6	Honest Responses & Errors	15+ honest HTTP status codes, `Location` on 201, unified RFC 7807 `problem()` helper, multi-field 422/400 validation array, masked 500s.	`api-gateway/src/problem.js`, `order-service/src/problem.js`.

End of Software Requirements Specification (SRS) — SmartCart Distributed E-Commerce Platform